

LAPORAN PROYEK

IMPLEMENTASI LOAD BALANCER BERBASIS

DOCKER

MENGGUNAKAN ALGORITMA ROUND ROBIN DAN

LEAST CONNECTION DENGAN REAL-TIME

DASHBOARD



Disusun Oleh:

Nama : Muhammad Faizal

NIM : 105841104023

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga laporan proyek yang berjudul "Implementasi Load Balancer Berbasis Docker Menggunakan Algoritma Round Robin dan Least Connection dengan Real-Time Dashboard" ini dapat diselesaikan dengan baik.

Proyek ini merupakan implementasi nyata dari konsep distribusi beban (load balancing) dalam sistem terdistribusi yang menggunakan teknologi modern seperti Node.js, Docker, dan WebSocket. Melalui proyek ini, penulis berupaya mendemonstrasikan cara kerja dua algoritma load balancing yang umum digunakan dalam lingkungan produksi.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang bersifat membangun sangat diharapkan demi penyempurnaan laporan ini. Semoga laporan ini bermanfaat bagi pembaca.

Demikian laporan proyek ini penulis susun. Atas perhatian dan dukungan dari semua pihak, penulis ucapkan terima kasih.

Makassar, Juni 2025

Muhammad Faizal

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI	3
BAB I PENDAHULUAN	5
1.1 Latar Belakang.....	5
1.2 Rumusan Masalah.....	5
1.3 Tujuan.....	6
1.4 Manfaat	6
1.5 Batasan Masalah.....	7
BAB II TINJAUAN PUSTAKA.....	8
2.1 Load Balancer	8
2.2 Algoritma Round Robin.....	8
2.3 Algoritma Least Connection.....	9
2.4 Docker dan Containerization.....	9
2.5 Node.js dan Express.....	9
2.6 WebSocket	10
BAB III METODOLOGI	11
3.1 Metode Pengembangan.....	11
3.2 Arsitektur Sistem.....	11
3.3 Alat dan Bahan	12
BAB IV IMPLEMENTASI	13
4.1 Struktur Proyek.....	13
4.2 Backend Server.....	13
4.3 Load Balancer Server	14
4.4 Konfigurasi Docker	15
4.5 Dashboard Web	15
BAB V HASIL DAN PEMBAHASAN.....	17
5.1 Cara Menjalankan Aplikasi	17
5.2 Pengujian Algoritma Round Robin.....	17
5.3 Pengujian Algoritma Least Connection	18
5.4 Perbandingan Algoritma.....	18
BAB VI PENUTUP.....	19
6.1 Kesimpulan	19

6.2 Saran.....	19
DAFTAR PUSTAKA	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat mendorong meningkatnya kebutuhan terhadap sistem yang andal, skalabel, dan mampu menangani beban kerja yang tinggi secara efisien. Dalam era digital saat ini, sebuah aplikasi web modern dituntut untuk mampu melayani ribuan bahkan jutaan permintaan secara bersamaan tanpa mengalami penurunan performa yang signifikan.

Load balancing merupakan salah satu teknik fundamental dalam arsitektur sistem terdistribusi yang digunakan untuk mendistribusikan beban kerja secara merata ke beberapa server backend. Dengan teknik ini, tidak ada satu server pun yang mengalami kelebihan beban (overload) sementara server lainnya menganggur, sehingga penggunaan sumber daya menjadi lebih optimal.

Docker sebagai platform containerization telah merevolusi cara pengembang membangun, mendistribusikan, dan menjalankan aplikasi. Dengan Docker, setiap komponen sistem dapat dikemas dalam container yang terisolasi, portabel, dan konsisten di berbagai lingkungan.

Proyek ini mengimplementasikan sebuah Load Balancer berbasis Docker menggunakan Node.js yang mendukung dua algoritma distribusi beban: Round Robin dan Least Connection. Dilengkapi dengan real-time dashboard berbasis web dan terminal logging berwarna, proyek ini memungkinkan observasi langsung terhadap perilaku kedua algoritma dalam mendistribusikan request HTTP ke tiga backend server.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah:

1. Bagaimana mengimplementasikan load balancer dengan algoritma Round Robin menggunakan Node.js dan Docker?

2. Bagaimana mengimplementasikan algoritma Least Connection sebagai alternatif distribusi beban yang adaptif?
3. Bagaimana memvisualisasikan distribusi request secara real-time melalui web dashboard berbasis WebSocket?
4. Bagaimana perbandingan kinerja algoritma Round Robin dengan Least Connection dalam kondisi beban bervariasi?

1.3 Tujuan

Tujuan dari proyek ini adalah:

1. Mengimplementasikan load balancer fungsional dengan dua algoritma distribusi beban menggunakan teknologi Node.js.
2. Mengorkestrasikan layanan (load balancer dan tiga backend server) menggunakan Docker dan Docker Compose.
3. Membangun real-time dashboard yang mampu menampilkan status distribusi request secara live menggunakan WebSocket.
4. Menganalisis dan membandingkan perilaku kedua algoritma dalam berbagai skenario pengiriman request.

1.4 Manfaat

Proyek ini diharapkan dapat memberikan manfaat sebagai berikut:

1. **Bagi Mahasiswa:** Memperoleh pemahaman praktis tentang konsep load balancing, containerization dengan Docker, dan komunikasi real-time dengan WebSocket.
2. **Bagi Akademisi:** Menyediakan referensi implementasi load balancer sederhana yang dapat digunakan sebagai bahan ajar dalam mata kuliah Sistem Terdistribusi atau Jaringan Komputer.
3. **Bagi Industri:** Menjadi bukti konsep (proof of concept) implementasi load balancer ringan yang dapat dikembangkan lebih lanjut untuk kebutuhan produksi skala kecil.

1.5 Batasan Masalah

1. Proyek ini hanya mengimplementasikan dua algoritma load balancing: Round Robin dan Least Connection.
2. Backend server yang digunakan bersifat simulasi dengan delay pemrosesan acak antara 20-80 ms.
3. Sistem dijalankan di lingkungan lokal (localhost) menggunakan Docker Compose dan tidak mencakup deployment ke cloud.
4. Tidak mencakup fitur health check otomatis, failover, atau SSL/TLS termination.

BAB II

TINJAUAN PUSTAKA

2.1 Load Balancer

Load balancer adalah komponen perangkat lunak atau perangkat keras yang berfungsi mendistribusikan lalu lintas jaringan atau beban kerja ke beberapa server backend. Tujuan utamanya adalah memastikan tidak ada satu server pun yang menjadi bottleneck, meningkatkan ketersediaan layanan (availability), dan memaksimalkan throughput sistem secara keseluruhan.

Menurut Tanenbaum dan Van Steen (2017), dalam sistem terdistribusi, load balancing merupakan salah satu mekanisme kunci untuk mencapai skalabilitas horizontal. Dengan menambah jumlah server dan mendistribusikan beban secara merata, sistem dapat menangani peningkatan jumlah pengguna tanpa harus meningkatkan kapasitas server tunggal secara vertikal.

Load balancer bekerja pada berbagai lapisan model OSI, mulai dari Layer 4 (Transport Layer) yang mendistribusikan berdasarkan informasi TCP/UDP, hingga Layer 7 (Application Layer) yang dapat membuat keputusan routing berdasarkan konten request HTTP seperti URL path, header, atau cookie.

2.2 Algoritma Round Robin

Round Robin adalah salah satu algoritma penjadwalan (scheduling) paling sederhana yang banyak digunakan dalam load balancing. Algoritma ini mendistribusikan request ke setiap server secara bergiliran dengan urutan yang tetap, sehingga setiap server menerima jumlah request yang relatif sama.

Secara matematis, jika terdapat N server dengan indeks 0 hingga $N-1$, maka server yang dipilih untuk request ke- i adalah $\text{server}[i \bmod N]$. Kompleksitas waktu algoritma ini adalah $O(1)$ karena tidak memerlukan pemeriksaan kondisi server.

Kelebihan Round Robin terletak pada kesederhanaannya dan distribusi yang merata secara statistik. Namun algoritma ini tidak mempertimbangkan perbedaan kapasitas server maupun beban kerja aktual yang sedang diproses.

2.3 Algoritma Least Connection

Algoritma Least Connection (LC) mendistribusikan request baru ke server yang saat ini memiliki jumlah koneksi aktif paling sedikit. Berbeda dengan Round Robin yang bersifat statis, Least Connection bersifat dinamis dan responsif terhadap kondisi beban server secara real-time.

Implementasi Least Connection memerlukan pemantauan terus-menerus terhadap jumlah koneksi aktif setiap server, sehingga kompleksitas waktunya adalah $O(n)$ di mana n adalah jumlah server. Algoritma ini memberikan distribusi yang lebih adil dalam skenario di mana setiap request memiliki durasi pemrosesan yang berbeda-beda.

2.4 Docker dan Containerization

Docker adalah platform open-source yang memungkinkan pengembang untuk mengemas, mendistribusikan, dan menjalankan aplikasi dalam container yang terisolasi. Container Docker ringan karena berbagi kernel OS host, namun tetap terisolasi satu sama lain dari segi proses, filesystem, dan jaringan.

Docker Compose adalah tool untuk mendefinisikan dan menjalankan aplikasi multi-container Docker. Dengan file konfigurasi YAML (`docker-compose.yml`), seluruh infrastruktur aplikasi dapat didefinisikan secara deklaratif.

2.5 Node.js dan Express

Node.js adalah runtime JavaScript berbasis event-driven dan non-blocking I/O yang dibangun di atas V8 JavaScript engine milik Google. Arsitektur event-loop Node.js menjadikannya sangat efisien untuk menangani banyak koneksi bersamaan (concurrent connections) dengan konsumsi memori yang rendah.

Express.js adalah framework web minimalis untuk Node.js yang menyediakan serangkaian fitur untuk membangun aplikasi web dan API RESTful.

2.6 WebSocket

WebSocket adalah protokol komunikasi yang menyediakan saluran komunikasi dua arah (full-duplex) melalui satu koneksi TCP. Berbeda dengan HTTP yang bersifat request-response, WebSocket memungkinkan server untuk mengirimkan data ke client kapan saja tanpa harus menunggu permintaan dari client.

Dalam proyek ini, WebSocket digunakan untuk menyiarkan (broadcast) pembaruan status server secara real-time ke semua client yang terhubung ke dashboard tanpa perlu polling dari sisi client.

BAB III

METODOLOGI

3.1 Metode Pengembangan

Proyek ini dikembangkan menggunakan pendekatan incremental, di mana sistem dibangun secara bertahap mulai dari komponen paling dasar hingga fitur yang lebih kompleks. Tahapan pengembangan meliputi:

1. Analisis kebutuhan dan perancangan arsitektur sistem.
2. Implementasi backend server dengan endpoint simulasi.
3. Implementasi load balancer dengan algoritma Round Robin.
4. Penambahan algoritma Least Connection.
5. Integrasi WebSocket untuk komunikasi real-time.
6. Pembangunan dashboard web dengan antarmuka visual.
7. Containerization menggunakan Docker dan Docker Compose.
8. Pengujian dan analisis hasil.

3.2 Arsitektur Sistem

Arsitektur sistem terdiri dari empat container Docker yang saling terhubung melalui Docker bridge network bernama lb-network:

Komponen	Port	Fungsi
Load Balancer	3000 (publik)	Menerima request dari browser, mendistribusikan ke backend, melayani dashboard
Backend server-1	3001 (internal)	Menerima dan memproses request dengan simulasi delay 20-80 ms
Backend server-2	3002 (internal)	Menerima dan memproses request dengan simulasi delay 20-80 ms
Backend server-3	3003 (internal)	Menerima dan memproses request dengan simulasi delay 20-80 ms

Load balancer hanya mengekspos port 3000 ke luar (host machine), sedangkan ketiga backend server hanya dapat diakses dari dalam Docker network. Hal ini meningkatkan keamanan sistem karena backend tidak dapat diakses langsung dari luar.

3.3 Alat dan Bahan

No	Alat/Teknologi	Versi / Keterangan
1	Node.js	Runtime JavaScript untuk load balancer dan backend server
2	Express.js	Framework web HTTP untuk Node.js
3	http-proxy	Library proxy HTTP untuk Node.js
4	ws (WebSocket)	Library WebSocket server untuk Node.js
5	uuid	Pembuatan request ID unik
6	Docker & Docker Compose	Platform containerization dan orkestrasi layanan
7	HTML/CSS/JavaScript	Teknologi frontend untuk dashboard web

BAB IV

IMPLEMENTASI

4.1 Struktur Proyek

Proyek ini memiliki struktur direktori sebagai berikut:

```
load-balancer-project/
├── docker-compose.yml
├── README.md
├── backend-server/
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
└── load-balancer/
    ├── Dockerfile
    ├── package.json
    ├── server.js
    └── public/
        └── index.html
```

4.2 Backend Server

Backend server diimplementasikan menggunakan Node.js dan Express. Setiap instance menjalankan kode yang sama tetapi dikonfigurasi dengan identitas berbeda melalui environment variable `SERVER_ID` dan `PORT`. Server menyediakan tiga endpoint utama:

Endpoint	Method	Deskripsi
GET /health	GET	Mengembalikan status kesehatan server, ID server, dan statistik koneksi
GET /ping	GET	Endpoint utama simulasi. Menambah koneksi aktif, memproses dengan delay acak 20-80ms, lalu mengurangi koneksi
GET /stats	GET	Mengembalikan statistik real-time: koneksi aktif dan total request

Cuplikan kode endpoint /ping pada backend server:

```
app.get('/ping', (req, res) => {
  activeConnections++;
  totalRequests++;
  const requestId = req.headers['x-request-id'] ||
  uuidv4();
  const delay = Math.floor(Math.random() * 60) + 20;
  setTimeout(() => {
    activeConnections--;
    res.json({ serverId: SERVER_ID, requestId,
      activeConnections, totalRequests, processingTimeMs:
    delay });
  }, delay);
});
```

4.3 Load Balancer Server

Komponen utama load balancer adalah sebagai berikut:

Server Pool: Tiga server backend dalam array SERVERS dengan properti id, url, activeConnections, dan totalRequests.

Round Robin: Variabel rrIndex sebagai pointer; dipilih dengan SERVERS[rrIndex % SERVERS.length].

Least Connection: Array.reduce() untuk menemukan server dengan activeConnections minimum.

API /api/algorithm: Endpoint POST untuk mengganti algoritma secara dinamis tanpa restart server.

API /api/test: Endpoint POST untuk mengirim batch request pengujian (maks 50) dengan jeda 30ms.

WebSocket Broadcast: Setiap perubahan status langsung disiaarkan ke semua client WebSocket via broadcast().

Implementasi fungsi pemilihan server:

```
function selectRoundRobin() {
  const s = SERVERS[rrIndex % SERVERS.length];
```

```

    rrIndex = (rrIndex + 1) % SERVERS.length;
    return s;
}

function selectLeastConnection() {
    return SERVERS.reduce((prev, curr) =>
        curr.activeConnections < prev.activeConnections ? curr
    : prev
    );
}

```

4.4 Konfigurasi Docker

File docker-compose.yml mendefinisikan empat layanan dalam satu stack. Semua layanan terhubung ke bridge network lb-network sehingga komunikasi antar container menggunakan nama container sebagai hostname.

1. Setiap backend server dibangun dari image yang sama tetapi dibedakan via environment variable SERVER_ID dan PORT.
2. Load balancer memiliki depends_on ketiga backend server untuk memastikan urutan startup yang benar.
3. Hanya port 3000 yang dipetakan ke host; backend server menggunakan expose (hanya internal network).
4. Semua service menggunakan restart: unless-stopped untuk pemulihan otomatis.

4.5 Dashboard Web

Dashboard web dibangun menggunakan HTML, CSS, dan JavaScript murni. Dashboard terhubung ke load balancer melalui WebSocket dan menampilkan informasi berikut secara real-time:

1. Status koneksi WebSocket (terhubung/terputus) dengan indikator berwarna.
2. Tombol pemilihan algoritma (Round Robin / Least Connection).
3. Kartu statistik setiap server: koneksi aktif dan total request.

4. Progress bar visual mencerminkan proporsi beban relatif antar server.
5. Log aktivitas request: ID, server tujuan, waktu respons, dan timestamp.
6. Kontrol pengiriman batch request dengan input jumlah (1-50).

BAB V

HASIL DAN PEMBAHASAN

5.1 Cara Menjalankan Aplikasi

Langkah-langkah menjalankan proyek:

Prasyarat: Pastikan Docker Desktop dan Docker Compose telah terinstal.

Clone Repository: Unduh atau clone repositori proyek ke direktori lokal.

Build dan Jalankan: Jalankan perintah: `docker-compose up --build`

Akses Dashboard: Buka browser dan akses `http://localhost:3000`

Pengujian: Gunakan tombol pada dashboard untuk memilih algoritma dan mengirim request.

Menghentikan: Tekan Ctrl+C atau jalankan: `docker-compose down`

5.2 Pengujian Algoritma Round Robin

Pengujian dilakukan dengan mengirim 10 request secara batch. Hasil menunjukkan distribusi request yang merata dan bergiliran ke ketiga server.

Request ke-	Server Tujuan	Keterangan
1	server-1	Request pertama
2	server-2	Bergiliran
3	server-3	Bergiliran
4	server-1	Siklus baru
5	server-2	Bergiliran
6	server-3	Bergiliran
7	server-1	Siklus baru
8	server-2	Bergiliran
9	server-3	Bergiliran
10	server-1	Siklus baru

Server-1 menerima 4 request (40%), server-2 dan server-3 masing-masing menerima 3 request (30%). Distribusi merata sesuai prinsip Round Robin.

5.3 Pengujian Algoritma Least Connection

Pengujian dilakukan dengan skenario yang sama (10 request, jeda 30ms). Karena delay pemrosesan bervariasi (20-80ms), distribusi tidak selalu bergiliran melainkan bergantung pada koneksi aktif saat itu.

1. Request 1: Semua server 0 koneksi aktif → server-1 dipilih.
2. Request 2 (30ms): server-1 masih memproses → server-2 dipilih.
3. Request 3 (60ms): server-1 dan server-2 masing-masing 1 koneksi → server-3 dipilih.
4. Request 4+: Bergantung server mana yang pertama selesai dan kembali ke 0 koneksi.

5.4 Perbandingan Algoritma

Aspek	Round Robin	Least Connection
Pola Distribusi	Bergiliran tetap (deterministik)	Berdasarkan beban aktual (dinamis)
Kompleksitas Waktu	$O(1)$	$O(n)$
Responsif thd Beban	Tidak	Ya
Overhead	Sangat rendah	Sedikit lebih tinggi
Cocok untuk	Beban seragam, request cepat	Beban bervariasi, request lama
Fairness	Merata secara jumlah	Merata secara beban
Predictability	Sangat mudah diprediksi	Bergantung kondisi runtime

Round Robin lebih sederhana dan ringan, cocok untuk skenario homogen. Least Connection lebih adaptif dan adil dalam skenario heterogen di mana request memiliki waktu pemrosesan yang sangat bervariasi.

BAB VI

PENUTUP

6.1 Kesimpulan

1. Implementasi load balancer berbasis Docker menggunakan Node.js berhasil dilakukan dengan dua algoritma: Round Robin dan Least Connection, berjalan dalam lingkungan terkontainerisasi menggunakan Docker Compose.
2. Algoritma Round Robin terbukti efektif untuk distribusi beban merata pada skenario homogen dengan kompleksitas $O(1)$.
3. Algoritma Least Connection menunjukkan keunggulan dalam skenario heterogen dengan mendistribusikan request ke server yang paling ringan bebannya secara real-time.
4. Integrasi WebSocket berhasil mewujudkan dashboard monitoring real-time tanpa perlu polling dari sisi client.
5. Docker dan Docker Compose berhasil menyederhanakan deployment dan memastikan konsistensi lingkungan antar komponen.

6.2 Saran

1. Menambahkan algoritma lain seperti IP Hash atau Weighted Round Robin untuk perbandingan yang lebih komprehensif.
2. Mengimplementasikan health check otomatis yang dapat mendeteksi server down dan mengeluarkannya dari pool.
3. Menambahkan fitur SSL/TLS termination pada load balancer untuk simulasi mendekati kondisi produksi.
4. Mengembangkan dashboard dengan grafik historis menggunakan library visualisasi seperti Chart.js.
5. Melakukan pengujian performa menggunakan Apache JMeter atau k6 untuk mengukur throughput secara kuantitatif.

6. Mengeksplorasi deployment ke platform cloud menggunakan Kubernetes sebagai orkestrator container.

DAFTAR PUSTAKA

- [1] Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms (3rd ed.). Pearson.
- [2] Docker Inc. (2024). Docker Documentation. Diakses dari <https://docs.docker.com>.
- [3] OpenJS Foundation. (2024). Node.js Documentation. Diakses dari <https://nodejs.org/docs>.
- [4] Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. RFC 6455. IETF.
- [5] Bourke, T. (2001). Server Load Balancing. O'Reilly Media.
- [6] Nginx Inc. (2024). NGINX Load Balancing Documentation. Diakses dari <https://docs.nginx.com>.
- [7] Express.js Contributors. (2024). Express.js Guide. Diakses dari <https://expressjs.com>.
- [8] Stallings, W. (2021). Operating Systems: Internals and Design Principles (10th ed.). Pearson.
- [9] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- [10] Richardson, C., & Smith, F. (2016). Microservices from Design to Deployment. NGINX, Inc.